Ingeniería del Software II

Unidad 1. Tema 4.

Diseño Orientado a Objetos.

Cómo Realizar el Diseño de Objetos a partir del Diagrama de Clases en UML

Trabajamos en un ejercicio y tratamos de cubrir estos tres aspectos:

1. Interpretar el Diagrama de Clases y sus características.
2. Realizar el diseño a partir del diagrama de Clases.
3. Diferenciar el diseño de objetos entre clases, relaciones, clase asociación y atributos de enlace.

El Caso de Estudio

- Nos solicitan desarrollar una aplicación que integre varias aplicaciones de mensajería Instantánea y sus contactos.

El Caso de Estudio

- Elementos principales.

. Nombre

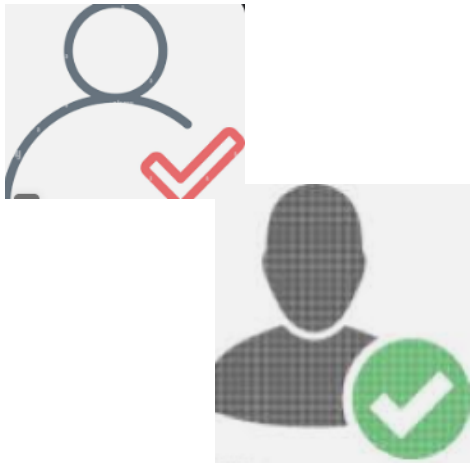
- Descripción
- URL acceso a la API

Dos tipos de usuarios

- Básico y validado.
- De ambos, conocer correo su electrónico.
- Del validado, registrar nombre y numero de teléfono. Podrá tener varias cuentas de mensajería.

Caso de Estudio.

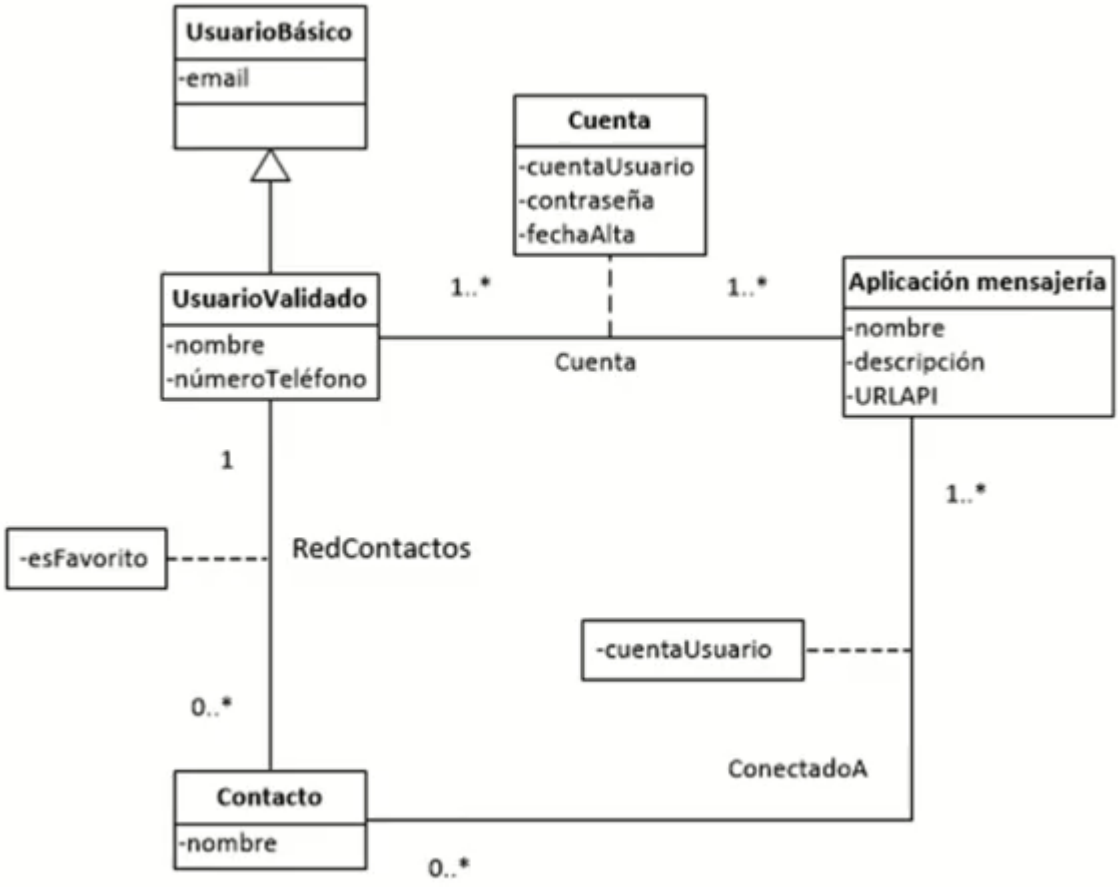
Usuario Validado



Podrá tener varias cuentas en las aplicaciones de mensajería y acceder a ellas mediante una cuenta de usuario y contraseña.
Se debe guardar la fecha de alta en la aplicación.
Fecha de alta, usuario y contraseña podrán variar en función de la aplicación.

Podrá tener una red de contactos.
El sistema debe permitir añadir cuentas en distintas aplicaciones de mensajería para cada contacto. Los contactos podrán marcarse como favoritos.

Diagrama de Clases.



Diseño de Objetos

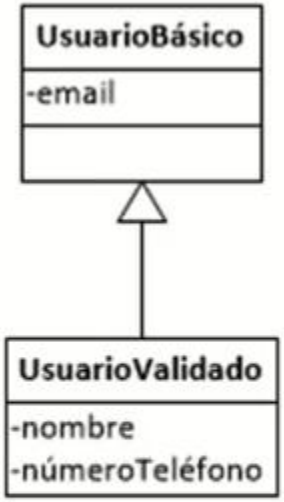
Diseño de los atributos de la clase como **propiedades** con su modo de acceso para lectura (**get**) y escritura (**set**)

```
public class UsuarioBasico
{
    0 referencias
    public virtual string email
    {
        get;
        set;
    }

}
```

```
public class UsuarioValidado : UsuarioBasico
{
    0 referencias
    public virtual string nombre
    {
        get;
        set;
    }

    0 referencias
    public virtual string numeroTelefono
    {
        get;
        set;
    }

}
```

Por defecto en C#

Diseño de Objetos



Diseño de los atributos de la clase como **atributos privados** (para que no sean accesibles desde fuera de la clase)

```
public class UsuarioBasico
{
    private string email;
}
```

Más los métodos get/set necesarios...

```
public class UsuarioValidado : UsuarioBasico
{
    private string nombre;
    private string numeroTelefono;
}
```

Gracias a la **herencia no repetimos atributos**

Continuamos con el Diseño

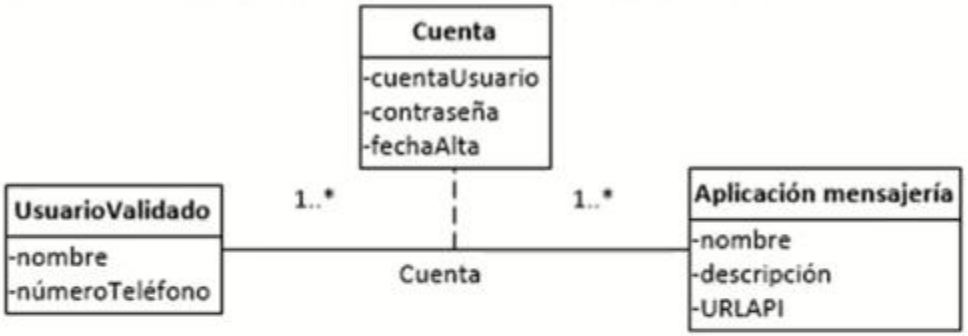
Análogamente con el resto de las clases…

```
public class AplicacionMensajeria
{
    private string nombre;
    private string descripcion;
    private string URLAPI;
}
```

```
public class Contacto
{
    private string nombre;
}
```

Relaciones y Atributos de Enlace

Análogamente con el resto de las clases...

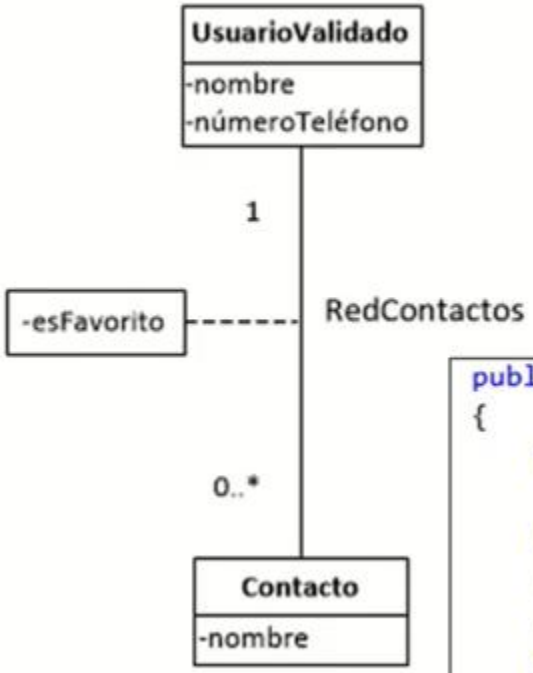



```
public class Cuenta
{
    private string cuentaUsuario;
    private string contraseña;
    private DateTime fechaAlta;

    // la clase asociacion necesita una referencia a las dos
    // clases de la relación, independientemente de la multiplicidad
    private UsuarioValidado usuarioValidado;
    private AplicacionMensajeria aplicacionMensajeria;
}
```

Cuenta es una **clase asociación**, por lo que **es una clase más**

Relaciones y Atributos de Enlace

Relaciones y atributo de enlace en relación 1-muchos




```
public class UsuarioValidado : UsuarioBasico
{
    private string nombre;
    private string numeroTelefono;
    // colocar esFavorito aquí sería un ERROR
    private ICollection<Contacto> redContactos;
}
```

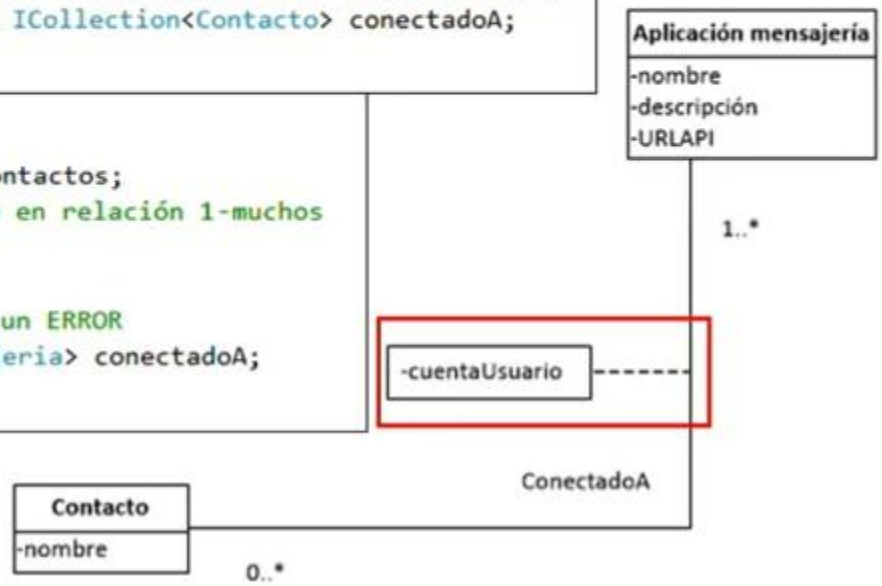
```
public class Contacto
{
    private string nombre;

    private UsuarioValidado usuarioRedContactos;
    // importante: el atributo de enlace en relación 1-muchos
    // se coloca en la parte de muchos
    private bool esFavorito;
}
```

Relaciones y Atributos de Enlace

Relaciones y atributo de enlace en relación muchos-muchos

```
public class AplicacionMensajeria
{
    private string nombre;
    private string descripcion;
    private string URLAPI;
    // colocar cuentaUsuario aquí sería un ERROR
    private ICollection<Contacto> conectadoA;
}
```

```
public class Contacto
{
    private string nombre;

    private UsuarioValidado usuarioRedContactos;
    // importante: el atributo de enlace en relación 1-muchos
    // se coloca en la parte de muchos
    private bool esFavorito;
    // colocar cuentaUsuario aquí sería un ERROR
    private ICollection<AplicacionMensajeria> conectadoA;
}
```

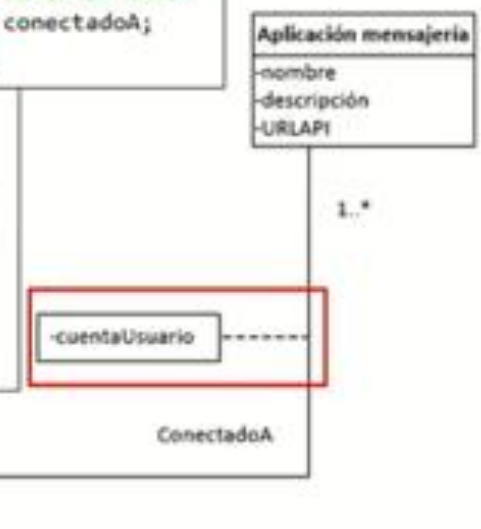


Relaciones y Atributos de Enlace

Relaciones y atributo de enlace en relación muchos-muchos

```
public class AplicacionMensajeria
{
    private string nombre;
    private string descripcion;
    private string URLAPI;
    // colocar cuentaUsuario aquí sería un ERROR
    private ICollection<Contacto> conectadoA;
}
```

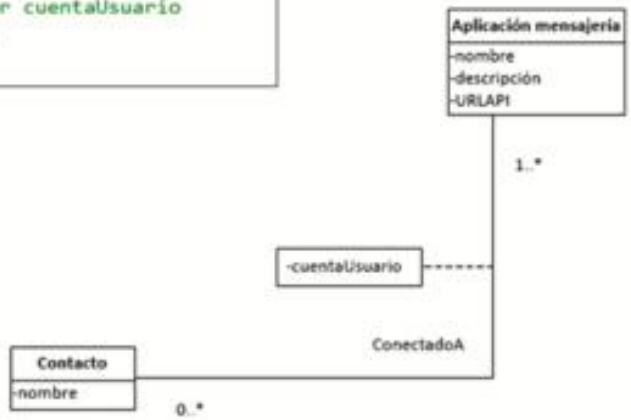
```
public class Contacto
{
    private string nombre;

    private UsuarioValidado usuarioRedContactos;
    // importante: el atributo de enlace en relación 1-muchos
    // se coloca en la parte de muchos
    private bool esFavorito;
    // colocar cuentaUsuario aquí sería un ERROR
    private ICollection<AplicacionMensajeria> conectadoA;
}
```



Relaciones y Atributos de Enlace

Relaciones y atributo de enlace en relación muchos-muchos

```
public class ConectadoA
{
    private Contacto conectadoAContacto;
    private AplicacionMensajeria conectadoAAplicacion;
    // la clase NO se puede llamar cuentaUsuario
    private string cuentaUsuario;
}
```



Relaciones y Atributos de Enlace

Relaciones y atributo de enlace en relación muchos-muchos

```
public class AplicacionMensajeria
{
    private string nombre;
    private string descripcion;
    private string URLAPI;
    // tener una colección de Contacto sería un error
    private ICollection<ConectadoA> conectadoA;
}
```

```
public class Contacto
{
    private string nombre;

    private UsuarioValidado usuarioRedContactos;
    // importante: el atributo de enlace en relación 1-muchos
    // se coloca en la parte de muchos
    private bool esFavorito;
    // tener una colección de AplicacionMensajeria sería un error
    private ICollection<ConectadoA> conectadoA;
}
```

Relaciones y Atributos de Enlace

Relaciones cuando se trata de clase asociación

```
public class UsuarioValidado : UsuarioBasico
{
    private string nombre;
    private string numeroTelefono;
    // colocar esFavorito aquí sería un ERROR
    private ICollection<Contacto> redContactos;
    // tener una colección de AplicacionMensajeria sería un error
    private ICollection<Cuenta> cuentas;
}
```

```
public class AplicacionMensajeria
{
    private string nombre;
    private string descripcion;
    private string URLAPI;
    // tener una colección de Contacto sería un error
    private ICollection<ConectadoA> conectadoA;
    // tener una colección de UsuarioValidado sería un error
    private ICollection<Cuenta> cuentas;
}
```

Conclusiones

- Derivar el diseño de objetos de un diagrama de clases es automático:
  - Aplico las pautas del diseño.
  - Las clases se diseñan siempre igual (aunque sea clase-asociación).
  - Las relaciones referencian a una única instancia o a una colección; dependiendo de la cardinalidad. No importa que sea una asociación, agregación o composición.
  - Los atributos de enlace deben promocionarse y diseñarse como clase-asociación cuando la cardinalidad es muchos a muchos.

Muchas
Gracias
Por
Su atención